



POLITECNICO
MILANO 1863



ITECNICO
LANO 1863

architettura dei calcolatori e sistemi operativi

espressioni algebriche

capitolo 2 P&H

espressione algebrica

il compilatore traduce in linguaggio macchina la valutazione di un'espressione algebrica affrontando in modo ottimizzato i seguenti passi schematici

- derivazione dell'albero sintattico dell'espressione
- ordinamento dei nodi dell'albero (ordine di calcolo)
- assegnamento dei registri alle variabili (allocazione dei registri)

noi adottiamo una «regola» semplice e intuitiva

che non richiede la derivazione dell'albero sintattico
e che non ottimizza la traduzione in modo completo



valutazione di espressioni algebriche

espressione: $a + b - 5 - (d - a) + c$

ma anche: $a + b - (\text{funz}(b - c, d) + c)$

procedimento (sulle slide delle convenzioni ci sono maggiori dettagli)

- completa l'espressione con le parentesi associando da sinistra
- ripeti fino ad avere esaminata tutta l'espressione
 - ✓ calcola il primo operatore valutabile (da sinistra verso destra)
 - ✓ un operatore è valutabile se le sue parti sinistra e destra sono: una variabile oppure una costante oppure una sottoespressione già calcolata in un registro di tipo t

Nota: le operazioni aritmetiche lavorano solo su registri

- ✓ se una variabile (tipicamente globale) è in memoria prima di usarla va (sempre) (ri)caricata in un registro (temporaneo); se è da aggiornare (p.es. per un assegnamento) va (ri)memorizzata
- ✓ se la variabile è associata ad un registro (argomento, variabile locale, ...) si usa il registro che la contiene; se è da aggiornare si modifica il registro



esempio 1 – variabili globali

espressione: $a + b - 5 - (d - a) + c$

- completa l'espressione con le parentesi associando da sinistra (potremmo anche omettere la parentesi più esterna)

$$\left(\left(\left(a + b \right) - 5 \right) - (d - a) \right) + c$$

- valuta l'espressione (risultato finale nel registro *t1*)

```
la    t0, A
ld    t1, 0(t0)    # carica A
la    t0, B
ld    t2, 0(t0)    # carica B
add   t1, t1, t2    # ( )
addi  t1, t1, -5    # ( )
la    t0, D
ld    t2, 0(t0)    # carica D
la    t0, A
ld    t3, 0(t0)    # carica A
sub   t2, t2, t3    # ( )
sub   t1, t1, t2    # ( )
la    t0, C
ld    t2, 0(t0)    # carica C
add   t1, t1, t2    # ( )
```

regole implicite utilizzate

- registro *t0* usato ripetutamente per indirizzi di variabili globali
- si riscrive il risultato nel registro con indice minore
- si riutilizzano i registri

ci sono un po' di gradi di libertà nella scelta dei registri e nel fino a che punto riutilizzarli (ottimizzazioni effettuate dal compilatore – qui non interessano)



esempio 2 – variabili globali e chiamata di funzione

variabili locali: sono allocate in pila ... vedi esempi nei TDE

espressione: $a + b - (\text{funz}(b - c, d) + c)$

- completa l'espressione con le parentesi associando da sinistra

$((a + b) - (\text{funz}(b - c, d) + c))$

- valuta l'espressione (ris. in $t1$ – sono sottointese tutte le **la** $t0$, **LABEL**)

ld	t1, 0(t0)	# carica A
ld	t2, 0(t0)	# carica B
add	t1, t1, t2	# ()
ld	t2, 0(t0)	# carica B
ld	t3, 0(t0)	# carica C
sub	a2, t2, t3	# primo parametro di funz
ld	a3, 0(t0)	# secondo parametro di funz
addi	sp, sp, -8	# push t1
sd	t1, 0(sp)	# salva t1 in pila
jal	FUNZ	# chiama funzione FUNZ
ld	t1, 0(sp)	# ripristina t1 da pila
addi	sp, sp, 8	# pop t1
ld	t2, 0(t0)	# carica C
add	t2, a0, t2	# ()
sub	t1, t1, t2	# ()

Nota Bene: *push* e *pop* di $t1$ per riaverlo inalterato dopo FUNZ.

